

vanilla.penplotter

HOW THE ENGINE WORKS, STAGE BY STAGE

From a drawing written in JavaScript to the stepper commands on the wire: the document model, the optimizer, the planner, the renderers, the EBB driver, and the decisions behind each of them. Written for anyone who wants to use the engine well, extend it, or check what it really does.

Sebastien Vanblaere · 2026-09-30 · MIT · seb-prjcts-be.github.io/vanilla.penplotter
· [PDF of docs/handbook.html](#)

Also available as a page on the site; this PDF is printed from it with `npm run handbook`.

CONTENTS

1. [The engine](#)
2. [Install and the first job](#)
3. [The document model](#)
4. [The facade: PlotterEngine](#)
5. [Geometry: primitives, fills, transforms, SVG import](#)
6. [The optimizer](#)
7. [The planner](#)
8. [The renderers](#)
9. [The EBB driver and its transports](#)
10. [Machine facts and safety rules](#)
11. [Plugins](#)
12. [Waves, series and other sources](#)
13. [The examples, one by one](#)
14. [Recipes](#)
15. [Tests, tooling and the site](#)
16. [Limits and roadmap](#)
17. [Reference tables](#)

1 THE ENGINE

A pen plotter is a machine that moves a pen over paper along lines you give it. The usual way to feed it from a generative sketch is a chain of four programs: record the sketch as an SVG, clean and sort the lines in vtype, open the file in Inkscape, send it with the plotter's own extension. Each step works. Together they lose what the sketch knew: which lines belong together, which pen they want, in which order they should be drawn.

`vanilla.penplotter` keeps that knowledge in one place. You build geometry in millimetres; the engine removes wasted travel, orders the paths, estimates the time, previews the result, and either streams the plan straight to a tested machine or writes the file another machine needs. It is a compiler for plotter jobs, with five stages that can each be used on their own:

Geometry	→	Optimizer	→	Planner	→	Renderer	→	Driver
paths		clean		order		preview		EBB commands
fills		merge		reverse		SVG		Web Serial
import		simplify		reloop		HPGL		dry run
transforms		resample		pens		G-code		safe stop

It is not a drawing library: no curves, no text, no booleans; draw elsewhere and hand over points. It is not `p5.plotSvg`, which captures p5 sketches as SVG and stops there, and not `vtype`, the mature command-line reference for line sorting. And it is not a universal driver: one machine profile has been plotted with, and the engine says so wherever it can.

TWO LIBRARIES, ONE DIRECTION

The engine knows nothing about p5.js. `p5.penplotter` adds three methods to p5 and calls this engine for everything else. Its own handbook covers that side. The dependency runs one way: the adapter checks the engine's version, never the reverse.

2 INSTALL AND THE FIRST JOB

The engine is a set of ES modules with no dependencies and no build step. The root module `vanilla.penplotter.js` imports from its own `src/` folder, so it has to be loaded from somewhere that serves the whole repository: a jsDelivr tag, GitHub Pages, or a local clone.

```
<script type="module">
  import { PlotterEngine } from "https://cdn.jsdelivr.net/gh/seb-prjcts-
be/vanilla.penplotter@v0.3.1/vanilla.penplotter.js";
</script>
```

The smallest complete job: a page, two shapes, a plan, and a look at the statistics.

```
const plot = new PlotterEngine({ units: "mm", page: { width: 210, height: 297, margin:
12 } });
plot.line(20, 30, 190, 30);
plot.circle(105, 145, 48);
const plan = plot.plan();
console.log(plan.stats); // { paths: 2, points: 98, drawDistance: ..., travelDistance:
..., penLifts: 2, ... }
```

From here the plan can go three ways: to a canvas with `plot.drawPreview(context)`, to a file with `plot.exportSVG()`, `exportHPGL()` or `exportGCode()`, or to the pen with the driver of chapter 9. Node 18 or newer is only needed to run the tests; the engine itself runs in any modern browser and in Node.

Everything the engine exports is reachable from the root module: the class `PlotterEngine`, the constant `VERSION`, every function of `src/core/model.js`, the class `PluginHost`, and the namespaces `Geometry`, `Optimizer`, `Planner`, `Renderer` and `Driver`. The driver module `src/driver/ebb.js` is also importable on its own, and that is how the examples do it.

3 THE DOCUMENT MODEL

Everything the engine works on is a plain object that survives `JSON.stringify`. That one rule makes the rest possible: stages can compare snapshots, return new copies, and never mutate what they were given. The authoring object is the *document*:

```
{
  schema: "vanilla.penplotter/document@1",
  units: "mm", // mm | cm | in | px
  page: { width: 210, height: 297, margin: 10, origin: "top-left" },
  tools: [ { id: "pen-1", name: "Black pen", color: "#111111", width: 0.35, kind:
"pen", metadata: {} } ],
  layers: [ { id: "default", name: "Default", toolId: "pen-1", visible: true, paths: [
... ], metadata: {} } ],
  metadata: {}
}
```

A *path* is a list of points with a few flags:

```
{ id: "path_12", points: [{ x, y }, ...], closed: false, reversible: true, metadata: {} }
```

- **Points** are `{ x, y }` in the document's unit. Every coordinate is checked to be a finite number when the path is created; `NaN` never gets in.
- **closed**: a closed path ends where it started. `createPath` appends a copy of the first point when a closed path does not already end there, so a closed path of n corners has $n + 1$ points.
- **reversible** (default `true`): the planner may draw this path from either end, and the optimizer may join it to a neighbour in either direction. Set it to `false` for a stroke whose direction matters, such as a brush stroke with a wet start.
- **id**: given or generated (`path_` plus a counter). Ids must be unique across layers and paths; `validateDocument` throws on a duplicate.
- **metadata**: anything serialisable. The fills put `effect: "hatch"` or `"stipple"` here; SVG import records the source tag; the optimizer records `mergedFrom`.

A *layer* groups paths and names one *tool* by id. A tool is a pen: `color` and `width` are used by the preview and the SVG; the id is what HPGL's pen map and the driver's pen-change pause refer to. A layer can be hidden with `visible: false`; the planner then skips it entirely.

Units are declared once, on the document, and everything follows: coordinates, tolerances, distance statistics. Four units are accepted. `px` is the CSS pixel, defined as 1/96 inch, and exists so a canvas-sized document can still be exported to physical sizes; direct plotting refuses it, because a pixel on a screen is not a length on paper.

UNIT	MILLIMETRES PER UNIT	WHERE IT MATTERS
mm	1	the default and the unit of every example
cm	10	accepted by the driver
in	25.4	accepted by the driver
px	$25.4 / 96 \approx 0.2646$	exports and the time estimate convert it; the driver refuses it

The functions in `src/core/model.js` build and check these objects: `createDocument`, `createLayer`, `createTool`, `createPath`, `addLayer`, `addPath`, `getLayer`, `cloneDocument`, `documentBounds`, `validateDocument`, plus the small helpers `point`, `clonePoint`, `distance`, `pathLength` and `millimetersPerUnit`. They are the vocabulary the other stages speak.

4 THE FACADE: PLOTTERENGINE

`PlotterEngine` is one object that holds a document, an active layer, and a cache of the last optimisation and the last plan. Beginners never need anything else; experts use it as a convenient front for the stages behind it.

METHOD	WHAT IT DOES
<code>new PlotterEngine(options)</code>	creates a document from <code>units</code> , <code>page</code> , <code>tools</code> , <code>metadata</code> , adds one layer (<code>layerId</code> , default <code>"default"</code> , <code>toolId</code> , default <code>"pen-1"</code>) and makes it active. <code>plugins</code> may pass a <code>PluginHost</code> .
<code>tool({ id, name, color, width, kind })</code>	adds a pen, or replaces the one with the same id. Returns the tool.
<code>layer(id, options)</code>	selects the layer with that id, creating it when needed (<code>toolId</code> , <code>name</code> , <code>paths</code>). The active layer receives all drawing calls that follow. Returns the layer.
<code>line</code> , <code>polyline</code> , <code>polygon</code> , <code>rect</code> , <code>circle</code> , <code>arc</code>	add one path to the active layer and return it. Options go to <code>createPath: id, closed, reversible, metadata</code> ; <code>circle</code> also takes <code>segments</code> .
<code>hatch</code> , <code>crossHatch</code> , <code>stipple</code>	fill a polygon with paths on the active layer and return the array of paths created.
<code>importSVG(text, options)</code>	parses an SVG and appends its layer(s) to the document; a clashing layer id gets a numeric suffix. Returns the imported document.
<code>optimize(options)</code>	runs the optimizer on the document, stores the result and the options, forgets any plan. Returns the optimized document.
<code>plan(options)</code>	plans the optimized document (optimizing first with the remembered or given <code>optimize</code> options if needed, or the raw document with <code>useRaw: true</code>), stores the plan and the options. Returns the plan.
<code>stats()</code>	the current plan's statistics.
<code>exportSVG</code> , <code>exportHPGL</code> , <code>exportGCode</code> , <code>exportJSON</code>	render the current plan; options go to the renderer.
<code>drawPreview(context, options)</code>	draws the current plan on a canvas 2D context.
<code>use(plugin)</code>	installs a plugin into the engine's host. Chainable.
<code>document</code> , <code>activeLayer</code> , <code>optimized</code> , <code>planned</code>	public fields. Editing <code>document</code> directly is allowed and detected.

HOW THE FACADE KNOWS THE PLAN IS STALE

There is exactly one mechanism, and it is deliberately blunt. Before any reuse of the cached optimisation or plan, the facade serialises the whole document to JSON and compares the string with the one it kept. Any difference, whether it came from a drawing method, from `tool()` or `layer()`, from `importSVG()`, or from a direct edit of `plot.document` or of a path or layer it returned earlier, drops both caches. The next call to `plan()`, `stats()`, an export or the preview rebuilds them with the last optimisation and planning options. A plan you were handed earlier is a separate object and is never touched.

The cost is one comparison per call: about 5 ms at 11 000 points, 20 ms at 50 000 and 230 ms at 500 000, measured on the development machine. For one plot or export that is nothing. Redrawing a very large plan as a live preview every frame is the one case where it shows; keep the plan object you were given and draw that instead. The rule this buys is worth it: no hidden state, no separate invalidation per method, and document data that must stay serialisable anyway.

5 GEOMETRY: PRIMITIVES, FILLS, TRANSFORMS, SVG IMPORT

PRIMITIVES

All primitives produce polylines; there are no curve objects in this version. A `circle` is a closed polygon of `segments` points, at least 12, by default 96. An `arc` takes a centre, a radius and two angles in radians and samples one point per 7.5 degrees ($\pi/24$), at least two segments. A `rectangle` refuses a zero or negative width or height. `polygon` is `polyline` with `closed: true`.

HATCH

`hatchPolygon(layer, points, { spacing, angle })` fills a simple polygon with parallel lines. The algorithm rotates the polygon by `-angle` so the hatch direction becomes horizontal, walks scanlines from `floor(minY / spacing) * spacing` to `maxY`, intersects each scanline with every edge (the half-open test `a.y ≤ y < b.y` avoids double hits at vertices), sorts the crossings, pairs them even-odd, and rotates each pair back into place as a two-point path tagged `effect: "hatch"`. Consequences worth knowing: a concave polygon is filled correctly; holes are not supported (pass the hole as its own polygon and it will be hatched too); the lines are separate paths, so the optimizer's merge pass and the planner's reversal are what turn them into a boustrophedon route. Spacing must be positive; the default is 2 mm and the default angle 45 degrees.

`crossHatchPolygon` is two hatch passes: `angle` and `angleB`, which defaults to `angle + 90°`.

STIPPLE

`stipplePolygon(layer, points, { count, minDistance, radius, seed, attempts })` scatters short dashes. A seeded generator (a 32-bit mixing function, the same algorithm as the examples' `randomFactory`) proposes points in the polygon's bounding box; a point is kept when it lies inside the polygon (ray casting) and no accepted point is closer than `minDistance` (default 1.2). It stops at `count` points (default 200) or after `attempts` proposals (default $40 \times \text{count}$, at least $20 \times \text{count}$). Each dot becomes a

horizontal dash of `2 × radius`, default 0.24 mm, because a pen cannot make a dot: it has to move to leave ink. Same seed, same dots, on every machine.

TRANSFORMS

A matrix is the SVG six-tuple `[a, b, c, d, e, f]`. `Matrix.translate`, `scale`, `rotate` (radians) and `multiply(a, b)` build them; `multiply(a, b)` means "apply b, then a", so to scale a drawing and then move it, write `multiply(translate(tx, ty), scale(s))`. `transformPoint` applies a matrix to one point; `transformDocument` returns a transformed copy of a whole document and leaves the original alone.

`offsetPolyline` is a naive normal offset that is fine for open sketch lines and wrong at cusps and self-intersections; the architecture document names the robust replacement it should get one day.

SVG IMPORT

`importSVG(text, options)` parses with the browser's `DOMParser` (in Node, pass one in `options.DOMParser`) and walks the element tree, multiplying `transform` attributes as it descends. Supported: `path` (M, L, H, V, C, S, Q, T, A, Z, absolute and relative), `line`, `polyline`, `polygon`, `rect`, `circle` and `ellipse`; transforms `matrix`, `translate`, `scale`, `rotate` (with or without a centre), `skewX`, `skewY`. Cubic and quadratic curves are flattened to `curveSteps` points each (default 16, at least 4); elliptical arcs follow the SVG centre-parameter conversion and are sampled at `arcSteps` per full turn (default 64); circles at `circleSegments` (default 96). Every path lands on one layer, `svg` by default, with `source: "svg"`, the tag and the element id in its metadata.

Two limits to keep in mind. The importer reads `width` and `height` as bare numbers and adopts `options.units`, `mm` by default; an SVG in pixels is therefore imported at one unit per pixel, and the `svg_to_pen` example fits it to a chosen width afterwards with a transform, which is the intended way. And styles are ignored on purpose: fills, stroke widths and colours do not survive, only centrelines, which is what a pen draws.

6 THE OPTIMIZER

`optimizeDocument(document, options, pluginHost)` returns a new document in which every visible layer's paths have gone through a list of passes. The default list is `["deduplicate", "merge", "simplify", "resample", "clean"]`; give `passes` to change the order, leave some out, or insert a plugin pass by name. An unknown name throws.

PASS	WHAT IT DOES	OPTION
<code>deduplicate</code>	drops a path whose point list, quantised to the tolerance, equals one already seen; for reversible paths the reversed list counts as the same.	<code>duplicateTolerance</code> , default 0.01
<code>merge</code>	joins two open, reversible paths whose ends lie within the tolerance, in any of the four end-to-end combinations, reversing one when needed; repeats until nothing joins. Records <code>mergedFrom</code> .	<code>mergeTolerance</code> , default 0.05
<code>simplify</code>	Ramer–Douglas–Peucker: removes points that deviate less than the tolerance from the line between their neighbours.	<code>simplifyTolerance</code> , default 0.03
<code>resample</code>	inserts points so no segment is longer than the limit. Off unless the option is given.	<code>maxSegmentLength</code>
<code>clean</code>	removes paths shorter than the minimum, except stipple dashes.	<code>minPathLength</code> , default 0.01
<code>name</code>	a plugin registered as an optimizer under that name: <code>(paths, options, layer) → paths</code> .	

Two details that came out of real drawings. The simplifier measures the distance to the finite segment, not to the infinite line through its ends; a polyline that goes out and comes back along the same line (a backtrack) would otherwise collapse to nothing. And the merge pass is quadratic in the number of paths per layer, which is fine for the thousands of hatch lines a sheet has and would not be for hundreds of thousands; the architecture document describes the spatial-hash version that would replace it.

The `route_lab` example is the optimizer's test bench: shuffled fragments, duplicates and a 1 440-point circle, with each pass switchable. On that input the passes together

take 141 paths and 2 141 points down to 15 paths and 464 points, before the planner even starts.

7 THE PLANNER

`planDocument(document, options)` turns a document into a *plan*: an ordered list of moves that the preview, the exports and the driver all consume, so they can never disagree about what will be drawn.

```
{
  schema: "vanilla.penplotter/plan@1",
  units, page, tools,
  routes: [ { layerId, toolId, path } ],           // the paths in drawing
  order
  moves: [ { type: "tool-change", toolId, layerId },
            { type: "travel", from, to, length, layerId, toolId },
            { type: "draw", points, length, pathId, layerId, toolId, metadata } ],
  stats: { paths, points, drawDistance, travelDistance, penLifts, penDowns,
  toolChanges, estimatedSeconds },
  options
}
```

Layers are planned in document order; a layer whose tool differs from the previous one emits a `tool-change` move first, so all work for one pen is grouped and the plot stops as few times as possible. Within a layer the strategy decides the order:

- **nearest** (default): greedy nearest neighbour from the current pen position, starting at `options.origin` (0,0). For an open, reversible path the closer of its two ends is chosen and the path is reversed when needed. For a closed path the seam is moved: the polygon is rotated so it starts at the corner nearest the pen (*reloop*), unless `reloop: false`. A path with `reversible: false` is only ever approached at its start.
- **input**: draw in the order given, no reversal, no reloop. Use it when the order is the drawing, or to see what the planner saves.

Every drawn path counts one pen-down and one pen-up. The time estimate, in seconds, gives every stroke and every hop its own ramps, with distances converted to millimetres first:

```
 $\Sigma \text{ramps}(\text{stroke}, \text{drawSpeed}, \text{acceleration}) + \Sigma \text{ramps}(\text{hop}, \text{travelSpeed}, \text{travelAcceleration}) + \text{penLifts} \cdot \text{liftDelay} + (\text{toolChanges} - 1) \cdot \text{toolChangeDelay}$   
 $\text{ramps}(L, v, a) = L / v + v / a \text{ when } L \geq v^2 / a, \text{ otherwise } 2 \cdot \sqrt{L / a}$   
defaults: drawSpeed 40 mm/s, travelSpeed 120 mm/s, acceleration 800 mm/s2,  
travelAcceleration 1200 mm/s2, liftDelay 0.6 s, toolChangeDelay 15 s
```

The defaults are the machine's: the speeds, ramps and pen delays of the EBB profile for the iDraw HSE / A2, so the estimate on an example page and the time in the pen panel agree, and so does the plot (wave hatch: 681 s estimated, 680 s plotted). The first tool is in the holder when the plot starts, so only a swap costs a tool change. With `acceleration: 0` the estimate is the plain distance over speed again. The planner never mutates its input and always returns fresh point objects, so a plan can be kept, compared or serialised while the document moves on.

8 THE RENDERERS

A renderer turns a plan into a machine program or a picture. They all read the same `moves` and `routes`, so the SVG, the HPGL and the preview show the same order and the same seams the pen will follow.

SVG

`renderSVG(plan, { precision, showTravel })` writes one `<g>` per layer with `id` and `data-tool`, one `<path>` per route with `d` as `M ... L ...`, `data-order` and `data-path`, and a `<metadata>` element per layer carrying the pen colour. The root has `width` and `height` in the document's unit and a matching `viewBox`, so the file opens at true size. Strokes are a uniform 0.35 hairline in black; `showTravel: true` adds the pen-up moves as dashed red paths for inspection. Coordinates are rounded to `precision` decimals, default 3.

HPGL

`renderHPGL(plan, { unitsPerMm, penMap })` emits `IN;SP1;`, then per route `PU x,y;PD x,y,x,y,...;PU;`, and `SP0;` at the end. Coordinates are integers in plotter units: `unitsPerMm` (default 40, the classic HP 0.025 mm) times the document's unit conversion. A `tool-change` move becomes `SPn`, where `n` comes from `penMap[toolId]` or, without a map, counts up from 1; give a map whenever the pen numbers matter. HPGL's y axis is not flipped; the plan's top-left origin is written as is.

G-CODE

`renderGCode(plan, { drawFeed, travelFeed, penUp, penDown })` writes `G21 G90`, then per route a pen-up, a rapid `G0` to the start, the pen-down, one `G1` per point, and a pen-up, ending with `M2`. Coordinates are in millimetres whatever the document unit; feeds are mm/min (defaults 1800 and 4200); the pen commands default to `M5` and `M3 S1000`, the servo idiom of GRBL pen plotters, and are options because every machine differs. Tool changes become comments.

JSON AND THE PREVIEW

`renderJSON` is the plan itself, pretty-printed unless `compact`. `drawPreview(context, plan, options)` fits the page into the canvas with `padding`, paints the paper (`paper`, default warm white), optionally draws the travel moves dashed (`showTravel`, `travelColor`), then every draw move in its tool's colour and width scaled to the canvas. `progress` between 0 and 1 draws only the first part of the plan, which is how a plotting animation would be made.

9 THE EBB DRIVER AND ITS TRANSPORTS

The EBB (EiBotBoard) is the controller in the AxiDraw and its relatives, including the iDraw HSE. It speaks a small text protocol over a USB serial port: a command, a comma-separated argument list, a carriage return; a reply ending in `OK`, or a line starting with `!` for an error. The driver uses nine of its commands:

COMMAND	MEANING	USED FOR
<code>V</code>	firmware version	the handshake: the reply must contain "EBB", or nothing is sent
<code>EM,1,1</code> / <code>EM,0,0</code>	enable both motors at 16× microstepping / disable them	first and last command of every plot
<code>SP,0,d</code> / <code>SP,1,d</code>	pen down / pen up, then wait <i>d</i> ms	every pen change; d is the profile's 300 ms
<code>LM,r1,a1,c1,r2,a2,c2,3</code>	low-level move: motor 1 takes <i>a1</i> steps starting at rate <i>r1</i> with acceleration <i>c1</i> per 40 μs tick, motor 2 likewise; 3 clears both accumulators first	every constant-acceleration phase of every stroke (firmware 2.7+)
<code>SM,ms,a1,a2</code>	move motor 1 by <i>a1</i> steps and motor 2 by <i>a2</i> steps in <i>ms</i> milliseconds, at constant speed	the same phases cut into slices of at most 25 ms, on firmware older than 2.7
<code>QU,2</code> / <code>CU,4,n</code>	read the largest possible motion queue / set the queue to <i>n</i> commands	once per plot on firmware 3.x, before the first move
<code>QM</code>	motion status	polled until idle before a pen change and before the motors are switched off
<code>ES</code>	emergency stop: clear the motion queue	the safe stop

COREXY

The iDraw, like the AxiDraw, is a CoreXY machine: two motors, one belt, and neither motor moves one axis alone. Moving both by the same amount moves the pen along X; moving them by opposite amounts moves it along Y. The mixing is therefore $a1 = (x + y) \cdot \text{steps/mm}$ and $a2 = (x - y) \cdot \text{steps/mm}$, and 80 steps per millimetre at 16× microstepping. This was not taken from a datasheet: on 2026-09-21 one motor alone drew a diagonal on paper, both together drew a line along X, and a 40 mm frame

measured 40 mm. The "113 steps/mm" an earlier test had measured was $80 \times \sqrt{2}$, a diagonal mistaken for an axis.

COMPILING A PLAN

`compileEbbPlan(plan, { profile, drawSpeed, travelSpeed, acceleration, travelAcceleration, junctionDeviation, commandSet, returnHome })` is pure: it turns a plan into the exact list of commands and sends nothing. Speeds come from the options, else from the plan's own options (what `plan({ drawSpeed })` was given), else from the profile; the other values from the options, else the profile. In order:

1. The plan's unit must be mm, cm or in, and every point of every travel and draw move, converted to millimetres, must lie inside the profile's travel (594×432 mm). Otherwise it throws, before anything exists to send.
2. `EM,1,1`, then pen up.
3. For each move: a `tool-change` after the first tool becomes a pen-up, a wait-until-idle marker and a marker entry the driver pauses on; a `travel` is a pen-up stroke; a `draw` is a pen-up stroke to its first point, pen down, and the whole path as one pen-down stroke.
4. Pen up, an optional stroke back to 0,0 (`returnHome`, default on), a wait-until-idle marker, `EM,0,0`.

THE SPEED PROFILE

A stroke is one polyline, travelled from rest to rest. Before planning, chords within `simplifyTolerance` (0.02 mm, under two steps) of a straight line are merged, so a circle drawn as 360 chords of 0.2 mm becomes a few dozen commands rather than 360 commands of 5 ms that the board cannot parse in time. `planStroke(points, settings)` then gives every vertex the speed it may be passed at: the floor `minSpeed` (2 mm/s) at both ends, never exactly zero, because a step rate of zero leaves the last step to rounding luck and the pen would sit on the paper waiting for it; and in between GRBL's *junction deviation* rule, $v^2 = a \cdot \delta \cdot \sin(\theta/2) / (1 - \sin(\theta/2))$, where θ is the turn at the vertex and δ the profile's 0.05 mm. A straight continuation never slows, a 45° bend passes at about 22 mm/s with the default 800 mm/s^2 , a right angle at about 10 mm/s, a reversal drops to the floor. A forward and a backward pass then cap every vertex speed by what the ramps can reach over the neighbouring segments ($v^2 \leq v_0^2$

+ 2 a L). Each segment becomes a trapezoid, accelerate, cruise, brake, or a triangle when it is too short to reach cruise speed; each of those phases is one constant-acceleration piece from one point to another with a start and an end speed. A phase shorter than 4 ms (a ramp from 39.9 to 40 mm/s, say) is not worth a command of its own: its neighbour takes over the distance and the speed makes that tiny jump at the boundary instead, so a long segment never produces a command the board cannot parse in time. The cruise speed is the plan's draw or travel speed, further limited so that neither motor exceeds the board's 24.995 steps per millisecond (on CoreXY a motor sees $|v_x| + |v_y|$).

Every phase is sent against *absolute* step targets: the phase's end position in steps is rounded, and the command carries the difference from the steps already commanded. Rounding therefore never accumulates; after any plan the step totals return exactly to where they started, which the tests check. With `commandSet: "LM"` (the default) a phase is one `LM`: duration in 40 μ s ticks from the average speed, and per motor an initial rate and an acceleration in the firmware's units ($2^{31} \times 40 \mu$ s per step/s), chosen so that the rate runs from the start speed to the end speed and the accumulator lands a quarter step past the step count, so rounding makes the last step fall a tick early rather than leave it hanging; both motors therefore finish together and consecutive commands hand the speed over without a jump. A test replays the firmware's own tick-by-tick arithmetic on every command to prove it. A motor with no steps gets zeros; a moving motor whose numbers round to nothing gets an acceleration of 1 so the firmware cannot stall; a phase so short that integer steps would push the rate past the limit is stretched by a tick. `LM` has no minimum rate, so the nearly-diagonal line that used to lose steps to the 1.31 steps/s floor now gets every step. With `commandSet: "SM"`, chosen by the driver for firmware older than 2.7, each phase is cut into slices of at most 25 ms, each a constant-speed `SM` at the speed the profile has at that moment: the ramps survive as a staircase, and a single step in 25 ms stays above the floor. The compiled object carries the profile id, the command set, the settings used, the commands with their kind and duration, statistics (`drawMm`, `travelMm`, `penDowns`, `durationMs`) and the end position.

RUNNING IT

`new EbbDriver({ transport, profile }).run(planOrCompiled, options)` refuses without `confirmed: true`: the caller states that the carriage is parked, the paper is in

place and hands are clear. It sends `v` and refuses a device that does not answer with an EBB version; the version decides the command set (a plan is compiled for it, a precompiled `LM` list is refused on firmware older than 2.7 with a message that says so). On firmware 3.x it asks the board for its largest motion queue (`QU,2`, 32 commands on the iDraw HSE) and opens it (`CU,4,n`); the profile's `fifoDepth` caps the request. Then it streams the commands: it keeps sending while at least two are still unacknowledged and the unacknowledged ones add up to less than the profile's `aheadMs` (250 ms) of motion, never more than the board's queue holds. The EBB answers when a command is queued, and with a full queue only after the oldest queued move has finished, so the acknowledgements are the flow control, a run of 5 ms moves is not paced by the USB round trip, and a stop stays quick because the `ES` behind the few commands in flight is parsed as soon as they are. Per command the timeout is 5 s plus the durations of everything that may still be queued. `onProgress(index, total, entry)` is called as each command is *acknowledged*, not executed; `onToolChange(toolId)` is awaited on a pen-change marker after the queue has drained, with the pen up and the motors holding, which is where a page asks you to swap pens. `abort()` sets a flag that the loop checks before every command. Abort and any error, including an `!` reply, end the same way: `ES`, `SP,1`, `EM,0,0`, each sent even if the previous one fails. The result is `{ status: "complete" | "aborted", commands, durationMs }`.

TRANSPORTS

A transport is only `open()`, `send(cmd, { timeoutMs }) → reply` and `close()`. Two exist.

`createLogTransport({ version, failOn })` is the dry run: it records every command in `log`, answers `v` and `QM` plausibly and everything else with an empty reply. Every example's "Dry run" button uses it, and so do the tests, including the ones that simulate a firmware error.

`createWebSerialTransport(port, { filters, baudRate })` is the browser transport. Without a port it calls `navigator.serial.requestPort`, which only works from a user gesture and only in Chrome and Edge on `localhost` or https; with a port granted earlier (from `navigator.serial.getPorts()`) it reuses it. It opens at 115200 baud, starts a reader loop that appends everything received to a buffer, and answers `send` by writing the command plus `\r` at once and handing out replies the moment their lines arrive (no polling timer): a line starting with `!` is an error reply, everything before an

`OK` line is the reply, and for `V` and `QM`, which firmware 3.x answers without `OK`, the first line is. Several commands may be in flight; the EBB answers in the order it receives, so replies go to the waiting callers in send order. A reply that never comes fails everything behind it, because the order would be lost. The test suite drives this transport with a fake port built from the same WHATWG streams a browser hands out, answering exactly as the machine did on 2026-09-21, including a reply that arrives in two chunks.

THE PEN PANEL

`examples/pen.js` is the reference consumer of all of the above and the panel every example page shows: it places the plan on the bed, turned by 0, 90, 180 or 270 degrees and moved so the sheet's corner lies at a chosen offset from home (`placePlan`, a pure function over the plan's moves, unit-aware, a rotation and never a mirror), draws the bed as the machine sees it (home in the corner by the board, X along the long rail, the sheet at its offsets, the drawing inside, so a portrait sheet that would land sideways is seen before it is plotted), compiles it, shows the command count and the machine's own time estimate, offers the dry run, connect, plot and stop, pauses for pen changes with a dialog, and keeps the SVG, HPGL and G-code exports as a side door. It also remembers, per drawing and in `localStorage`, how many strokes an interrupted plot had completed (each pen-up in the compiled list carries `completes`, the index of the stroke it ends), guarded by a signature of the drawing, and offers "Resume at stroke n": a compile with `skipDraws: n` leaves those strokes out and starts from home, where you park the carriage again by hand. While a plot runs, `lockLinks` greys out every link that leaves the page (navigation, GitHub, the rest of the site; the panel's own download links stay) and restores them afterwards. Leaving the page anyway gets the browser's "leave site?" question first; if the page goes anyway, `driver.emergencyStop()` writes `ES`, `SP,1` and `EM,0,0` in one go without waiting for replies, so the pen comes up. That gap was found the hard way on 2026-10-01, with a half-finished plot on the paper. The panel contains no protocol knowledge; everything it needs is in the driver.

10 MACHINE FACTS AND SAFETY RULES

PROFILE IDRAW-HSE-A2	VALUE	SOURCE
board and firmware	EBB, firmware 3.0.2 (3.0.3 exists; it only changes error messages)	read from the machine
USB ids	vendor 0x04d8, product 0xfd92, 115200 8N1	Windows device manager; used as the port filter
kinematics	CoreXY, motor 1 = X + Y, motor 2 = X - Y	drawn on paper on 2026-09-21
resolution	80 steps/mm at 16× microstepping (2032 steps/inch)	40 mm frame measured; vendor's <code>axidraw_conf.py</code>
travel	594 × 432 mm	vendor configuration for the A2 model
step rate	maximum 24.995 steps/ms, minimum 1.31 steps/s per motor	EBB firmware limits
speeds	40 mm/s drawing, 120 mm/s travelling; the plan's own speeds win	chosen under the vendor's defaults (55 and 166 mm/s)
acceleration	800 mm/s ² drawing, 1200 mm/s ² travelling, junction deviation 0.05 mm	chosen under the vendor's 40 in/s ² (1016 mm/s ²)
floor and merge	minimum pen speed 2 mm/s (<code>minSpeed</code>); chords within 0.02 mm of straight merged (<code>simplifyTolerance</code>)	chosen: a zero rate leaves the last step hanging; the board parses a command in a few ms
motion queue	up to 32 commands on firmware 3.x (<code>fifoDepth</code>); 250 ms of motion sent ahead of the acknowledgements (<code>aheadMs</code>)	asked from the board with <code>QU,2</code> ; chosen
pen timing	300 ms after each pen up or down	chosen
origin	the carriage position when the plot starts; there is no homing	EBB has no end stops

The rules the driver enforces, and the ones only you can: park the carriage in the home corner by hand before every plot, because 0,0 is wherever the carriage is. Call `open()` from a click. Run a dry run the first time a plan changes shape. Keep the confirmation dialog honest. Keep hands, cables and paper magnets out of the arm's path. And for now, one machine: everything in this chapter and the one before it is tested on the iDraw HSE / A2 only, with EBB firmware 3.0.2; the axes and the scale on 2026-09-21, the acceleration planning, the `LM` moves and the deep motion queue on 2026-10-01.

The profile table has entries for AxiDraw, generic iDraw and GRBL machines as data for page sizes and exports, but the only driver claim is the one that has been plotted with. An AxiDraw speaks the same protocol and should behave the same; until someone plots with one, that is a belief, not a test.

11 PLUGINS

A `PluginHost` keeps named implementations in four registries: `optimizer`, `effect`, `renderer` and `driver`. A plugin is an object with `install(host)` that calls `host.register(type, name, fn)`; registering the same name twice throws. The engine's `use(plugin)` installs into its own host, and the optimizer looks up any pass name it does not know in the host's optimizer registry. That is the only extension point wired in today; the other three registries exist so that effects, renderers and drivers can be added the same way later.

```
const weave = {
  install(host) {
    host.register("optimizer", "weave", (paths, options, layer) =>
paths.slice().reverse());
  }
};

plot.use(weave);
plot.optimize({ passes: ["deduplicate", "merge", "weave"] });
```

Plugins receive snapshots and return new arrays; they never see the engine's private state. Because their input is plain data, a heavy pass could run in a worker without changing the contract.

12 WAVES, SERIES AND OTHER SOURCES

The engine wants points and nothing else, so anything that gives a number for an x is a drawing. `Math.sin` works; so does a noise function; so do `p5.waves` and `vanilla.waves`, whose `createSampler({ wave, range, frequency })` returns an object with `sample(x, t)`. The one idea to hold on to when plotting something that animates: pick a time and keep it. Every wave example on the site samples at `t = 0`; sample at ten different times and you have a series of ten sheets from one sketch.

```
const sampler = waves.createSampler({ wave: "triangle sine", range: [-3, 3], frequency:
0.08 });
for (let row = 0; row < 32; row += 1) {
  const points = [];
  for (let x = 18; x <= 192; x += 0.5) points.push({ x, y: 40 + row * 7 +
sampler.sample(x + row * 2, 0) });
  plot.polyline(points);
}
```

A series is the same loop one level up: one engine per sheet, a seed per sheet, and the same file name with a number. There is no state shared between engines, so this is safe and fast.

13 THE EXAMPLES, ONE BY ONE

Every example on the site ends at the pen through the shared panel of chapter 9, and every one that has no hardware dependency also runs headlessly in the test suite through its composition module.

EXAMPLE	WHAT IT SHOWS	FILES
<code>direct_plot</code>	a 40 × 25 mm frame with two sine periods: the first thing to plot, and a scale check (measure the frame)	<code>sketch.js</code>
<code>first_job</code>	hatch, outline and circle; travel toggle; the smallest complete job	<code>composition.js</code> , <code>sketch.js</code>
<code>two_pens</code>	two layers with two tools; the pen-change pause; HPGL pen map	<code>composition.js</code> , <code>sketch.js</code>
<code>route_lab</code>	deliberately bad input; every optimizer pass switchable; before/after statistics	<code>composition.js</code> , <code>sketch.js</code>
<code>waves_pen</code>	32 rows from <code>vanilla.waves</code> at a fixed time	<code>drawing.js</code> , <code>sketch.js</code>
<code>wave_hatch</code>	seventy cells hatched at a spacing and angle two waves decide: tone from lines, about ten metres of ink	<code>drawing.js</code> , <code>sketch.js</code>
<code>svg_to_pen</code>	paste or pick an SVG, fit it to a width, plan it, plot it	<code>sketch.js</code>

The split between a composition module and a sketch is the convention borrowed from `p5.gysin`: the composition only builds geometry and returns the engine; the sketch owns the DOM, the statistics and the pen panel. The gallery page imports the compositions to draw its live previews, and the test suite imports them to prove they build a plan.

14 RECIPES

TONE FROM LINES

```
const cell = (x, y, s) => [{ x, y }, { x: x + s, y }, { x: x + s, y: y + s }, { x, y: y + s }];
[6, 4, 2.6, 1.7, 1.2].forEach((spacing, i) => plot.hatch(cell(20 + i * 30, 40, 26), {
  spacing, angle: Math.PI / 4 }));
plot.crossHatch(cell(170, 40, 26), { spacing: 1.2 }); // darker still
```

TWO PENS, ONE STOP

```
plot.tool({ id: "black", color: "#111", width: 0.3 });
plot.tool({ id: "red", color: "#c8402c", width: 0.8 });
plot.layer("outline", { toolId: "black" }); plot.circle(105, 148, 60);
plot.layer("fill", { toolId: "red" });      plot.hatch(square, { spacing: 2 });
// the driver pauses once, between the two layers; HPGL gets SP1 and SP2 with:
plot.exportHPGL({ penMap: { black: 1, red: 2 } });
```

ANY SVG, FITTED AND PLOTTED

```
const imported = Geometry.importSVG(svgText, { curveSteps: 24 });
const b = documentBounds(imported);
const s = 120 / (b.maxX - b.minX); // 120 mm wide
const fitted = Geometry.transformDocument(imported,
  Geometry.Matrix.multiply(Geometry.Matrix.translate(-b.minX * s, -b.minY * s),
    Geometry.Matrix.scale(s)));
for (const layer of fitted.layers) plot.layer(layer.id).paths = layer.paths;
```

A DRY RUN IN NODE

```
import { EbbDriver, createLogTransport } from "../src/driver/ebb.js";
const dry = createLogTransport();
await new EbbDriver({ transport: dry, profile: "idraw-hse-a2" }).run(plot.plan(), {
  confirmed: true });
console.log(dry.log.length, "commands;", dry.log.slice(0, 5));
```

A PLOTTING ANIMATION

```
let progress = 0;
function frame() {
  progress = Math.min(1, progress + 0.004);
  Renderer.drawPreview(context, plan, { progress, showTravel: false });
  if (progress < 1) requestAnimationFrame(frame);
}
frame();
```

15 TESTS, TOOLING AND THE SITE

`npm test` runs four Node scripts with no framework: `tests/snapshot.js` (the pipeline end to end, the fills, the facade's staleness rule for every editing method and every consumer, SVG import through an injected parser, the plugin host, the renderer, every local link on the site, the freshness of the generated pages, every example composition, and every live preview of the site drawn into a recording canvas), `tests/ebb.js` (CoreXY mixing, the command sequence, rounding drift, step-rate limits, bounds and units, the driver's run, abort and error paths, and the Web Serial transport against a fake port), `tests/regressions.js` (bugs that once happened: edits after planning, options surviving edits, units, backtracking) and `tests/version.js` (every place that states the version agrees). `tests/waves-integration.js` is an opt-in check against a downloaded vanilla.waves build, so the suite stays off the network.

`npm run docs` renders `docs/architecture.md` and `docs/roadmap.md` to HTML pages in the site's chrome with `tools/build-docs.js`, a small Markdown converter without dependencies; the test fails when a page is stale. `npm run manifest` regenerates `docs/vanilla.penplotter.manifest.json`, the machine-readable summary other tools read. `npm run handbook` prints this handbook to PDF with a headless Chrome. The GitHub Actions workflow runs the tests and refuses a commit whose generated files are out of date. The site is served straight from `main` by GitHub Pages; there is no build.

The *Requirements* table is identical in this README and in p5.penplotter's, and a test in the adapter's repository compares the two blocks byte for byte. Publishing order is always the engine first, then the adapter, because the adapter's examples load the engine as a sibling folder and its version check would otherwise refuse the mismatch.

16 LIMITS AND ROADMAP

- **Geometry:** polylines only; curves are flattened at import; the offset is naive; no booleans, no clipping, no holes in fills.
- **Optimizer:** exact and reversed duplicates only, no partial overlap; merge is quadratic.
- **Planner:** greedy nearest neighbour; no 2-opt, no time-based cost, no layer dependencies.
- **Driver:** no pause or resume from a checkpoint; one tested profile; no Node transport yet, so plotting needs a browser; the acceleration profile is per stroke, so a pen-up travel always comes to rest before the pen goes down.
- **Facade:** the whole-document comparison costs milliseconds on huge documents.

Everything in this handbook is *sheet mode*: draw everything, then plot. The second mode, *live mode* (draw something, plot it: a session on the driver that takes one stroke at a time, in the order the sketch made them, so you can draw without a preview, draw with your hands through ml5, or watch a slow animation appear on paper), is designed on the [Two modes](#) page and not yet built. When it lands it gets its own chapter here.

The roadmap's direction is fixed: strengthen the direct route. Acceleration landed in 0.3.0; next are pause and resume, pen heights from the profile, a Node transport, and reading p5.gysin's plotter-SVG with its pen map. Parked, because vtype and the AxiDraw software already cover them: a curve-rich document, full SVG/CSS, robust offsets, a production planner, and drivers for machines nobody here can test on. Version 1.0 means stable schemas, TypeScript declarations, npm, and a support matrix where every profile has been plotted with.

17 REFERENCE TABLES

OPTIMIZER OPTIONS

OPTION	DEFAULT	UNIT
<code>passes</code>	deduplicate, merge, simplify, resample, clean	names
<code>duplicateTolerance</code>	0.01	document units
<code>mergeTolerance</code>	0.05	document units
<code>simplifyTolerance</code>	0.03	document units
<code>maxSegmentLength</code>	off	document units
<code>minPathLength</code>	0.01	document units

PLANNER OPTIONS

OPTION	DEFAULT	UNIT
<code>strategy</code>	nearest	nearest input
<code>reloop</code>	true	move closed-path seams
<code>origin</code>	{ x: 0, y: 0 }	document units
<code>drawSpeed</code> / <code>travelSpeed</code>	40 / 120	mm/s, for the estimate and the EBB driver
<code>acceleration</code> / <code>travelAcceleration</code>	800 / 1200	mm/s ² , ramps in the estimate; 0 for a plain sum
<code>liftDelay</code> / <code>toolChangeDelay</code>	0.6 / 15	seconds; a lift is pen down and up, a swap is every tool after the first
<code>useRaw</code>	false	plan the unoptimized document

RENDERER OPTIONS

RENDERER	OPTIONS	DEFAULTS
SVG	<code>precision</code> , <code>showTravel</code>	3, false
HPGL	<code>unitsPerMm</code> , <code>penMap</code>	40, none (pens count up from 1)
G-code	<code>drawFeed</code> , <code>travelFeed</code> , <code>penUp</code> , <code>penDown</code>	1800, 4200, M5, M3 S1000
JSON	<code>compact</code>	false
preview	<code>padding</code> , <code>paper</code> , <code>showTravel</code> , <code>travelColor</code> , <code>progress</code> , <code>ink</code>	24, warm white, false, red, 1, near black

DRIVER OPTIONS

WHERE	OPTION	DEFAULT
<code>compileEbbPlan</code>	<code>profile</code> , <code>drawSpeed</code> , <code>travelSpeed</code> , <code>acceleration</code> , <code>travelAcceleration</code> , <code>junctionDeviation</code> , <code>minSpeed</code> , <code>simplifyTolerance</code> , <code>commandSet</code> , <code>returnHome</code> , <code>skipDraws</code>	idraw-hse-a2, the plan's or 40 mm/s, the plan's or 120 mm/s, 800 mm/s ² , 1200 mm/s ² , 0.05 mm, 2 mm/s, 0.02 mm, LM, true, 0
<code>driver.run</code>	<code>confirmed</code> , <code>onProgress</code> , <code>onToolChange</code> , plus the compile options above when given a plan	required, none, none
<code>createWebSerialTransport</code>	<code>filters</code> , <code>baudRate</code> ; <code>send</code> takes <code>timeoutMs</code>	the profile's USB ids, 115200, 3000 ms
<code>createLogTransport</code>	<code>version</code> , <code>fifoMax</code> , <code>failOn</code>	EBB 3.0.2, 32, none

SCHEMAS

`vanilla.penplotter/document@1` (chapter 3), `vanilla.penplotter/plan@1` (chapter 7), `vanilla.penplotter/ebb@1` (the compiled command list, chapter 9). The adapter validates the plan schema before drawing; a plan from another version would be refused in plain words rather than drawn wrong.

This handbook describes version 0.3.1 as tested on 2026-10-01. Where the code and this text disagree, the code and its tests are right, and the text needs a fix: open an issue on GitHub.

[vanilla.penplotter](#) · [Handbook](#) · [About](#) · [p5.penplotter](#) for p5.js

MIT License · v0.3.1